

班 级 2203054w

学 号 22009201185

西安电子科技大学

本科毕业设计论文



题 目 智能客服机器人的设计与交互优化

学 院 计算机科学与技术学院

专 业 计算机科学与技术

学生姓名 王奇

导师姓名 魏昱恒、Harris H M XIE

毕业设计（论文）诚信声明书

本人声明：本人所提交的毕业论文《智能客服机器人的设计与交互优化》是本人在指导教师指导下独立研究、写作的成果，论文中所引用他人的无论以何种方式发布的文字、研究成果，均在论文中加以说明；有关教师、同学和其他人员对本文的写作、修订提出过并为我在论文中加以采纳的意见、建议，均已在我的致谢辞中加以说明并深致谢意。

本论文和资料若有不实之处，本人承担一切相关责任。

论文作者：_____（签字） 时间： 年 月 日

指导教师已阅：_____（签字） 时间： 年 月 日

摘要

随着银行业务线上化和服务渠道智能化的发展，传统人工客服在高频、重复业务中面临响应压力大、服务效率不稳定等问题。本文设计并实现了面向个人银行业务的智能客服机器人原型系统 BankCopilot。系统采用 Vue 3、Element Plus、ECharts、Spring Boot、MyBatis 和 MySQL 构建前后端业务能力，并接入通义千问大模型完成自然语言意图解析与分析性文本生成。系统实现了 AI 查余额、AI 查流水、AI 账单统计与分析、AI 周报/月报以及 AI 智能转账等核心功能。设计上，大模型只负责理解输入和组织表达，账户查询、流水查询、统计计算和转账执行均由后端确定性服务完成。测试结果表明，系统能够支持文本自然语言形式的银行客服交互，并在安全性、可解释性和可演示性方面具有一定实用价值。

关键词： 智能客服机器人；自然语言处理；银行业务；账单分析；交互优化

ABSTRACT

With the development of online banking services and intelligent service channels, manual customer service faces increasing pressure in high-frequency and repetitive scenarios. This thesis designs and implements BankCopilot, an intelligent customer service robot prototype for personal banking services. The system uses Vue 3, Element Plus, ECharts, Spring Boot, MyBatis and MySQL to build front-end and back-end business capabilities, and uses the Qwen large language model for natural language intent parsing and analytical response generation. The implemented AI functions include balance inquiry, transaction inquiry, bill statistics and analysis, weekly or monthly brief generation, and intelligent transfer confirmation. The model is responsible for understanding input and organizing natural language expressions, while account query, transaction query, statistical calculation and transfer execution are completed by deterministic back-end services. The test results show that the system can support text-based natural language banking service interaction and has practical value in security, explainability and demonstration.

Keywords: intelligent customer service robot; natural language processing; banking service; bill analysis; interaction optimization

目录

摘 要	i
ABSTRACT	iii
目录	v
第一章 绪论	1
1.1 研究背景与意义	1
1.2 课题目标与研究内容	2
1.3 本文组织结构	3
第二章 相关技术与理论基础	5
2.1 大语言模型与自然语言意图识别	5
2.2 Spring Boot 后端技术	6
2.3 Vue 与前端交互技术	6
2.4 数据持久化与统计分析技术	7
第三章 系统需求分析	9
3.1 系统总体需求	9
3.2 AI 核心功能需求	10
3.3 非功能性需求	10
第四章 系统总体设计	13
4.1 系统架构设计	13
4.2 AI 模块设计	13
4.3 数据库设计	15
4.4 接口与交互设计	16
第五章 AI 智能客服核心功能设计与实现	19
5.1 AI 请求处理总体流程	19
5.2 AI 查余额功能实现	21
5.3 AI 查流水功能实现	21
5.4 AI 账单统计与分析功能实现	23
5.5 AI 周报/月报功能实现	26
5.6 AI 智能转账功能实现	27
第六章 系统测试与交互优化分析	31
6.1 测试环境与测试方法	31

6.2 功能测试	32
6.3 交互优化设计分析	33
6.4 测试结论	34
第七章 结论与展望.....	37
7.1 主要结论	37
7.2 实践价值	37
7.3 不足与展望	38
结束语	39
致谢	41
参考文献	43
附录 A 主要接口与数据结构.....	45
A.1 AI 聊天请求结构	45
A.2 AI 聊天响应结构	45
A.3 统计查询结构	45

第一章 绪论

1.1 研究背景与意义

在数字化服务持续发展的背景下，银行业务从传统柜台办理逐步转向移动端、网页端和智能终端。用户在办理业务时，常常需要查询余额、查询交易流水、了解一段时间内的收支状况，或者发起转账等操作。这些需求频率高、重复性强，若完全依赖人工客服，不仅会增加银行服务成本，也会影响用户问题响应速度。因此，构建能够理解自然语言、连接业务系统并提供即时服务的智能客服机器人，具有较强的工程意义和应用价值。

近年来，大语言模型在自然语言理解和文本生成方面表现出显著能力，使客服机器人从传统的基于规则或检索的问答系统逐步发展为能够处理开放式表达和多轮上下文对话的智能助手^[1-3]。但在银行场景中，单纯依赖大模型直接回答并不可靠。银行业务涉及账户余额、交易流水和资金转账等敏感数据，要求系统具备准确性、可审计性和安全性。因此，本课题将大语言模型作为自然语言入口和表达增强工具，将真实业务查询、统计计算和资金操作交由后端确定性服务完成，从而在智能化体验和业务安全之间取得平衡。

BankCopilot 项目以银行智能客服为场景，实现了一个面向个人账户业务的 AI 助手。系统支持用户使用文本自然语言发起查询或操作请求，能够完成余额查询、流水查询、账单统计、周报/月报生成以及智能转账确认等功能。与普通客服问答系统相比，本系统的特点在于 AI 回复并非仅来自静态问答库，而是通过意图解析后调用真实账户、流水和统计服务，再将结果组织为用户可理解的文本、图表和报表导出链接。

从银行业务应用角度看，智能客服机器人并不只是一个问答窗口，而是业务入口、交互流程和风险控制逻辑的组合。用户在传统页面中完成一次流水查询，通常需要进入交易流水页面、选择交易类型、设置起止日期、调整分页参数，再阅读表格结果；而在智能客服交互中，用户可以直接表达“查最近 5 条支出记录”或“看最近 7 天支出趋势”。系统需要将这类口语化表达转换为后端能够执行的查询条件。该转换过程既涉及自然语言理解，也涉及业务规则约束。若只关注文本生成能力而忽略业务执行过程，智能客服就容易停留在演示层面，难以成为真正可用的银行服务入口。

另一方面，银行业务对准确性和安全性要求较高。余额、流水和转账结果均

属于用户关心的敏感信息，系统不能让模型根据语言习惯自行推断或编造金额。本文的设计思路是将大模型放在“理解用户意图”和“组织分析语言”的位置，将数据库查询、统计聚合和资金变动放在传统后端服务中执行。这样既可以利用大模型提升自然语言交互的灵活性，又可以通过确定性服务保证数据来源可追溯、业务流程可控制。该思路也是本文相对于简单聊天机器人实现的主要区别。

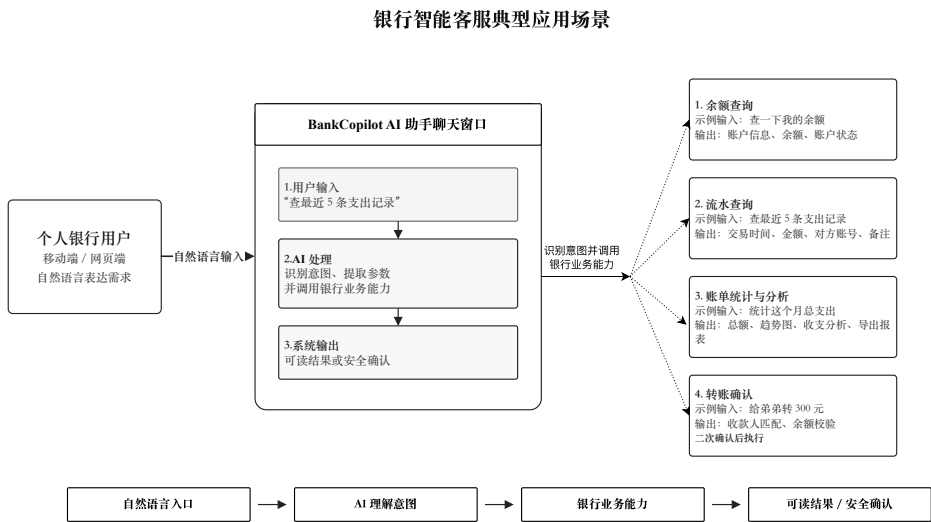


图 1.1 银行智能客服应用场景示意图

1.2 课题目标与研究内容

本课题的总体目标是设计并实现一个面向银行业务的智能客服机器人原型，重点研究自然语言交互、业务意图解析、对话状态管理、银行业务服务集成和前端交互优化等内容。系统以文本自然语言交互为主要方式，围绕用户在个人银行业务中的高频需求提供智能化辅助。

结合任务书要求和项目源码实现，本文的主要研究内容包括以下几个方面。

第一，设计智能客服机器人的功能体系和交互流程，明确 AI 助手支持的业务边界。第二，选择适合项目的自然语言处理方案，通过通义千问大模型完成用户输入到结构化意图的转换。第三，设计对话管理机制，实现转账确认和流水查询补全等多轮交互。第四，将 AI 助手与后端账户、流水、统计和转账服务集成，实现真实业务数据驱动的智能回复。第五，优化前端交互展示，提供流式响应、处理过程追踪、图表展示、快捷操作和报表导出等能力。

表 1.1 课题研究内容与系统实现对应关系

研究内容	本文实现方式
自然语言理解	通过 QwenClient 调用通义千问，将用户输入解析为 intent、时间范围、金额、收款人等结构化字段。
对话管理	通过 pendingMap 和 pendingTxnQueryMap 维护待确认转账和待补全流水查询状态。
业务系统集成	调用 AccountService、TransactionService、StatisticsService 和 PayeeService 完成真实业务处理。
交互优化	前端 AssistantPanel 实现 SSE 流式响应、AI 过程展示、图表展示、快捷确认与导出功能。
安全控制	转账前校验账户、余额和收款人信息，并要求用户二次确认。

上述研究内容共同构成了系统的 AI 能力闭环。自然语言理解负责把用户原始输入转换为可执行的业务意图，对话管理负责处理缺失信息和高风险操作确认，业务系统集成负责保证返回数据来自真实数据库，交互优化负责将处理过程和业务结果以更清晰的形式展示给用户。本文后续章节围绕这一闭环展开论述，避免把 AI 功能写成脱离系统实现的概念性描述。

课题研究范围需要明确边界。系统中的登录、注册，帐户验证等能力属于基础设施，不作为本文 AI 核心功能的重点，故不在本课题研究范围内。本文重点分析源码中已实现的 AI 查余额、AI 查流水、AI 账单统计与分析、AI 周报/月报和 AI 智能转账功能等 AI 集成特色功能。对于当前源码没有实现的消费品类识别、商户标签分析等能力，由于实现过于复杂且与 AI 集成不可控，仅在不足与展望中作为后续方向说明。

1.3 本文组织结构

本文共分为七章。第一章介绍课题背景、研究意义、研究目标和论文结构。第二章介绍系统开发涉及的关键技术，包括大语言模型、Spring Boot、Vue、MyBatis、MySQL、ECharts 和 SSE 流式通信等。第三章对系统需求进行分析，重点说明 AI 智能客服的核心功能需求和非功能性需求。第四章给出系统总体设计，包括系统架构、AI 模块、数据库和接口设计。第五章详细阐述 AI 查余额、AI 查流水、AI 账单统计与分析、AI 周报/月报和 AI 智能转账五项核心功能的设计与实现。第六章从功能测试和交互优化角度分析系统效果。第七章总结全文工作，并提出后续优化方向。

第二章 相关技术与理论基础

2.1 大语言模型与自然语言意图识别

自然语言意图识别是智能客服系统的入口能力，其目标是从用户的自然语言表达中识别用户想要完成的任务，并抽取执行任务所需的关键槽位信息。传统方法通常采用规则匹配、关键词识别或监督学习分类模型。随着大语言模型的发展，模型能够更好地理解口语化表达和复杂句式，适合用于将用户输入转换为结构化的意图表示。

本系统通过 QwenClient 封装通义千问兼容 OpenAI Chat Completions 的调用方式。在 AssistantService 中，parseIntentByQwen 方法将用户输入和系统提示词发送给模型，要求模型返回严格 JSON。该 JSON 包含 intent、toAccountNo、payeeAlias、amount、description、txnType、timeRange、metric、groupBy 等字段。后端随后根据 intent 分发到不同业务处理方法。

需要强调的是，在银行业务场景下，大语言模型无法直接作为业务事实来源。本系统将大模型限定为意图解析器和分析文本生成器，余额、流水、统计值和转账执行均来自后端业务服务。这种设计能够保留自然语言交互的灵活性，同时降低模型幻觉带来的业务风险。

在银行智能客服场景中，意图识别的目标不是单纯判断用户属于哪一个聊天主题，而是把自然语言输入转换为后端服务能够理解的参数组合。例如“这个月花了多少钱”需要被转换为统计意图、支出方向、按月时间范围和求和指标；“给弟弟转 300 元”需要被转换为转账意图、收款人昵称和金额。意图解析结果越接近后端接口参数，业务链路越短，系统越容易验证和维护。因此本文将模型输出限定为结构化 JSON，并在服务层进行二次校验。

大语言模型还承担分析性表达生成任务。对于统计结果，后端已经计算出总额、趋势和对比数据，直接返回数字给用户并不利于理解，因此，系统会把统计事实整理后交给模型生成自然语言总结，使回复更符合客服场景的表达习惯。需要强调的是，模型生成的分析只能基于后端提供的事实数据，不能扩展出数据库中不存在的交易类型、商户类别或风险结论。

2.2 Spring Boot 后端技术

Spring Boot 是系统后端的主体框架,用于快速构建 RESTful Web 服务^[4]。项目后端以 Controller、Service、Mapper 分层组织代码。其中 AssistantController 提供 AI 相关接口,AssistantService 完成 AI 请求编排,AccountService、TransactionService、StatisticsService 和 PayeeService 提供账户、流水、统计和收款人相关业务能力。

后端同时使用 Spring MVC 拦截器实现 Token 校验,通过 TokenInterceptor 解析 JWT 并将当前用户 id 保存到 CurrentHolder。AI 接口在调用业务服务前会读取当前用户 id,确保查询和操作均在登录用户上下文中执行。虽然登录鉴权不是本文重点,但它是 AI 查询个人账户数据和执行转账操作的必要基础。

从系统稳定性角度看, Spring Boot 后端承担了业务可信边界的角色。大模型服务可能因为网络、接口限流或返回格式异常而产生不确定性,但后端服务必须保证接口输入、权限边界、金额计算和数据库更新的确定性。AssistantService 在调用模型之后,会根据解析结果进入固定分支,再调用相应业务服务。这样的设计使模型能力被控制在服务编排层,而不会绕过后端已有的校验逻辑。

2.3 Vue 与前端交互技术

前端采用 Vue 3 构建组件化界面,使用 Element Plus 提供表单、按钮、抽屉、弹窗、表格等基础组件^[5]。AI 助手界面主要由 AssistantPanel.vue 实现,该组件同时用于首页主面板和右上角智能客服抽屉。

在 AI 交互方面,前端通过 fetch 调用 /assistant/chat/stream 接口,读取服务端发送事件流,并根据 trace、message、error、done 等事件更新界面。对于统计分析结果,前端根据 chartType 和 chartData 使用 ECharts 渲染柱状图、折线图、饼图和概览图^[6]。对于转账确认,前端根据 AI 回复内容显示“确认”和“取消”快捷按钮,从而降低用户输入成本。

前端交互设计直接影响智能客服的可用性与交互性。若系统只在页面底部显示一段长文本,用户很难判断 AI 是否真正调用了业务服务,也难以理解统计结果的口径。因此 AssistantPanel 在展示回复的同时,还展示处理过程、图表、快捷按钮和导出入口。对于答辩展示场景,这些可视化元素也能更直观地呈现系统的 AI 能力,而不是停留在控制台或接口返回层面。

2.4 数据持久化与统计分析技术

系统使用 MySQL 存储用户、账户、收款人和交易流水数据。MyBatis 用于完成 SQL 映射^[7]，TransactionMapper.xml 中定义了流水分页查询、总金额统计、笔数统计、最大交易查询、趋势聚合、收入支出对比、按对方账号聚合以及历史转账流水匹配等 SQL。

统计分析功能由 StatisticsServiceImpl 负责，根据前端或 AI 解析出的 StatisticsQueryDTO 选择 sum、count、max、trend 等统计方式。对于支出金额，SQL 中使用 ABS(amount) 将支出负数转为正向统计值，便于用户理解。统计结果再由 AssistantService 组织为事实文本，交给大语言模型生成分析总结，或者在模型不可用时使用规则化兜底总结。

数据持久化层为多项 AI 功能提供事实依据。余额查询依赖账户表，流水查询和统计分析依赖交易流水表，智能转账依赖常用收款人表和账户表，历史金额复用依赖历史交易记录。不同功能虽然在前端都表现为聊天输入，但在后端对应的数据读取和更新路径并不相同。论文在设计与实现章节中将分别说明这些路径，体现该系统不是简单拼接模型接口，而是围绕银行业务数据建立完整功能链路。

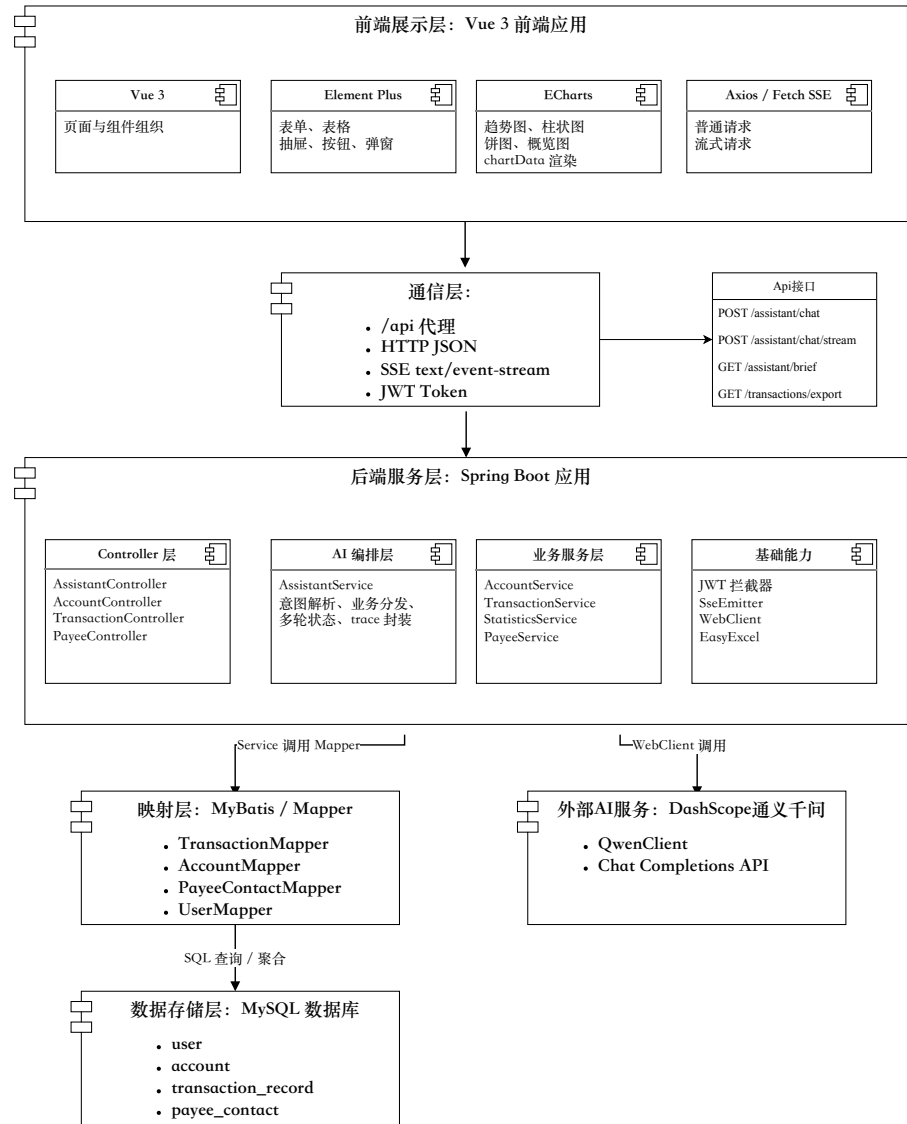


图 2.1 系统技术栈结构图

第三章 系统需求分析

3.1 系统总体需求

BankCopilot 面向银行个人用户，目标是通过智能客服机器人降低用户完成常见银行业务的操作复杂度。用户不需要记忆具体菜单和筛选条件，而是可以使用自然语言表达需求。系统应能够识别用户意图，将其映射到后端业务服务，并以文本、图表或导出文件等形式返回结果。

从用户角度看，系统需要支持账户余额查询、交易流水查询、收支统计分析、周报/月报查看和转账确认等能力。从系统角度看，AI 助手需要具备意图识别、槽位抽取、时间解析、多轮状态管理、业务服务调用、结果格式化和前端可视化展示等能力。

总体需求还体现在系统边界的划分上。AI 助手需要为用户提供更自然的入口，但不能替代后端业务规则。用户输入越口语化，系统越需要在后端建立明确的转换规则和校验逻辑。例如，用户可以说“这个月花了多少”，但后端必须把它转换为确定的起止时间、交易方向和统计指标；用户可以说“给弟弟转 300”，但后端必须识别收款人、校验金额、检查余额并等待确认。只有把自然语言表达和确定性业务规则结合起来，系统才能在银行场景中具备可用性。

此外，系统需满足本科毕业设计展示和答辩评阅的需要，在查看系统时，不仅需要看到 AI 回复文本，还需要看到它调用了什么业务能力、依据了哪些数据、是否存在安全确认环节。因此，系统在需求层面就需要考虑处理过程展示、统计图表展示和快捷操作按钮。它们不是额外装饰，而是证明系统具有真实业务链路的重要交互证据。

3.2 AI 核心功能需求

表 3.1 AI 核心功能需求表

功能编号	功能名称	需求描述
F1	AI 查余额	用户通过自然语言查询账户余额和账户基本信息，系统返回户名、账号、账户类型、状态和余额。
F2	AI 查流水	用户通过自然语言查询交易记录，系统支持收入/支出筛选、时间范围和查询条数解析。
F3	AI 账单统计与分析	用户查询总金额、笔数、最大交易或趋势时，系统基于数据库统计结果生成文本分析和图表数据。
F4	AI 周报/月报	用户打开 AI 面板时，系统展示本周或本月收入、支出、净流入、趋势、对比和异常提醒。
F5	AI 智能转账	用户通过自然语言发起转账，系统解析收款人、金额和备注，校验后等待用户确认再执行。

除上述核心功能外，系统还需要提供 AI 基础设施能力，包括自然语言意图识别、时间范围纠偏、SSE 流式响应、AI 处理过程展示、统计口径解释和 Excel 导出链接生成等。这些能力本身不是用户直接提出的业务目标，但对提升交互体验和系统可解释性具有重要作用。

3.3 非功能性需求

安全性方面，系统必须保证用户只能查询自己的账户和流水数据，资金流动操作必须经过校验和二次确认。AI 转账不能在用户首次输入后直接扣款，必须先生成确认信息，待用户回复“确认”后才能执行。对方账号在聊天回复中应进行脱敏展示。

准确性方面，系统中的账户余额、流水记录和统计金额必须来自数据库查询结果，不得由大模型直接编造。模型输出的统计总结应受到后端事实约束，若模型返回不合适内容，系统需要使用规则化总结兜底。

可用性方面，系统需要支持口语化输入，如“查一下我的余额”“看最近 7 天支出趋势”“给弟弟转 300 元”等。对于缺失关键信息的请求，系统应通过多轮对话引导用户补全，而不是直接报错。

交互性方面，AI 助手应尽量减少用户等待感和理解成本。系统通过流式响应展示处理过程，通过图表展示统计结果，通过快捷按钮完成转账确认或简报追问，从而提高交互效率。

可维护性方面，系统应保持前后端职责清晰。前端负责交互展示和用户输入收集，后端负责业务判断和数据访问，大模型调用被封装在独立客户端中。这样

做的好处是，当后续需要更换模型、调整提示词或增加统计指标时，可以优先修改 QwenClient、AssistantService 或 StatisticsService，而不需要大范围改动前端页面和数据库结构，清晰的模块划分有利于该系统设计的后续维护与迭代升级。

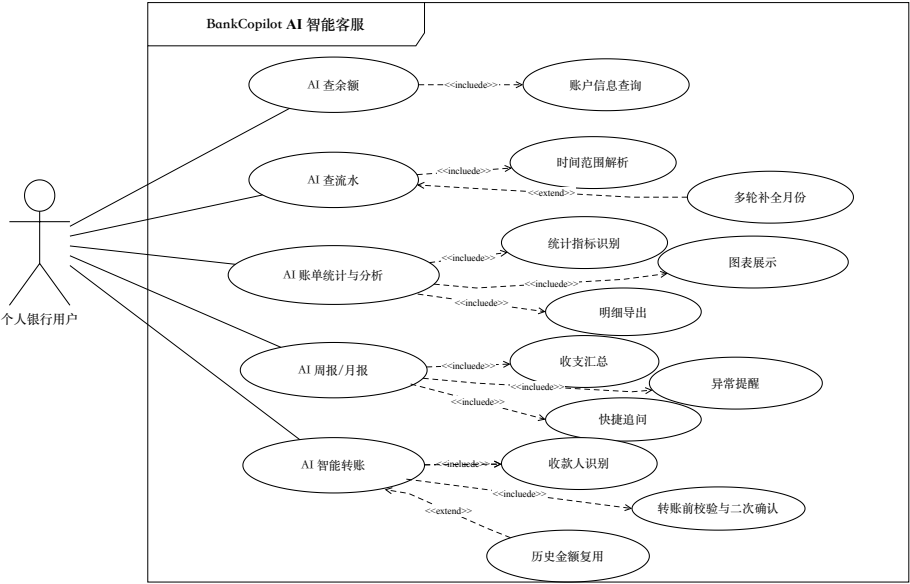


图 3.1 AI 智能客服用例图

第四章 系统总体设计

4.1 系统架构设计

系统采用前后端分离架构。前端由 Vue 3 应用负责页面展示和用户交互，后端由 Spring Boot 应用负责业务处理和数据访问，MySQL 负责数据持久化，DashScope 通义千问负责自然语言意图解析和分析性回复生成。前端通过 /api 代理访问后端接口，后端通过 WebClient 访问大模型服务。

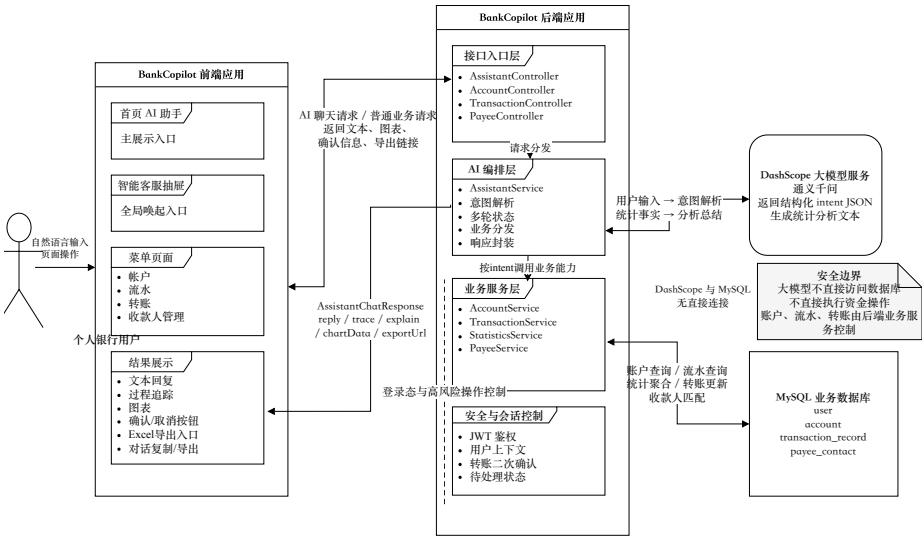


图 4.1 系统总体架构图

在该架构中，AI 助手并不是独立于业务系统之外的聊天模块，而是业务系统的一层自然语言交互入口。用户先通过自然语言输入页面操作，在前端应用的智能助手接入到后端应用的接口入口层，进行请求分发后进入 AI 编排层，通过 DashScope 进行意图分析出意图 intent，通过 intent 分发业务，再调用帐户，流水，统计或收款人服务，查询 MySQL 数据库，将结果封装后返回前端。需要强调的是，大模型无法直接访问数据库，不执行资金操作，帐户流水，转账有后端业务控制。

4.2 AI 模块设计

AI 模块由接口层、编排层、模型调用层、业务服务层和前端展示层组成。接口层对应 AssistantController，提供 /assistant/chat、/assistant/chat/stream 和

/assistant/brief 三类接口。编排层对应 AssistantService，负责处理待确认状态、调用 parseIntentByQwen、分发不同 intent、封装 trace 和 explain。模型调用层由 QwenClient 实现，统一封装通义千问请求。业务服务层包括 AccountService、TransactionService、StatisticsService 和 PayeeService。前端展示层由 AssistantPanel 负责。

这种模块划分的核心思想是“模型不直接接触数据库，业务不依赖模型自由发挥”。AssistantService 处于 AI 模块的中心位置，它既要理解模型返回的意图字段，又要根据业务规则决定是否继续执行。对于普通查询，服务层可以直接调用账户或流水服务；对于缺少月份的流水查询，服务层会保存待补全状态并追问用户；对于转账请求，服务层会生成待确认对象并等待用户下一轮确认。不同业务路径虽然都从聊天输入开始，但后端处理策略并不相同。

AI 模块还承担结果适配工作。业务服务返回的通常是对象、列表或聚合数据，而用户更希望看到简明文本、图表和下一步操作入口。因此 AssistantService 会把业务结果转换为 AssistantChatResponse，前端再根据 chartType、chartData、trace、explain 和 exportUrl 等字段决定如何展示。该设计使回复内容和展示形式解耦，可以在不破坏原有接口的基础上扩展，增加新的图表类型或统计指标。

表 4.1 AI 模块与源码对应关系

模块	源码位置	主要职责
AI 接口入口	AssistantController.java	接收聊天请求、流式聊天请求和简报请求。
AI 编排服务	AssistantService.java	意图解析、业务分发、多轮状态、统计总结。
模型调用	QwenClient.java	调用通义千问 Chat Completions 接口。
时间处理	Time.java	自然语言时间纠偏和时间范围填充。
前端面板	AssistantPanel.vue	聊天、简报、图表、过程追踪和确认按钮展示。

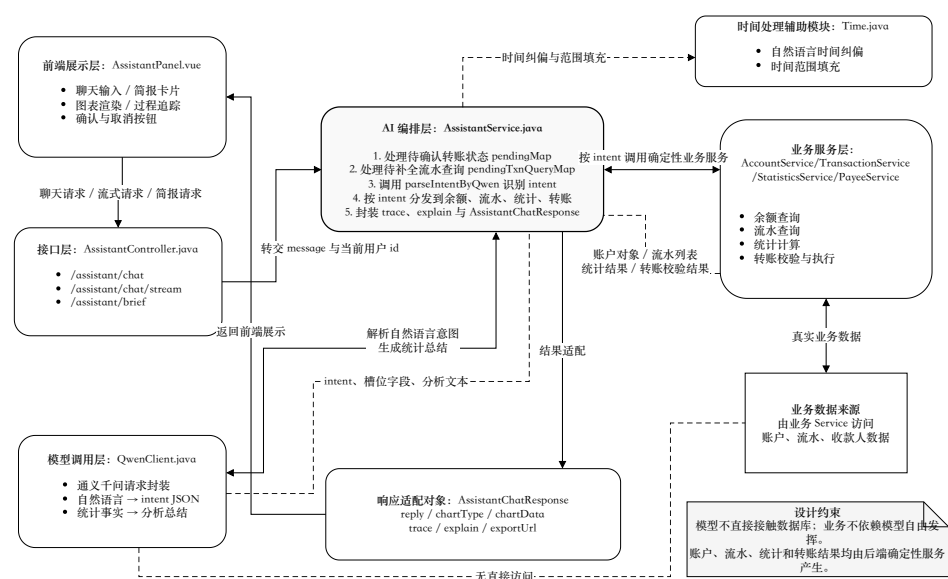


图 4.2 AI 请求处理架构图

4.3 数据库设计

系统 AI 功能依赖的核心数据表包括 `account`、`transaction_record`、`payee_contact` 和 `user`。AI 查余额主要读取 `account` 表和 `user` 表；AI 查流水和账单统计主要读取 `transaction_record` 表；AI 智能转账需要读取 `account` 表、`payee_contact` 表，并在执行后写入 `transaction_record` 表。

表 4.2 AI 功能相关数据表说明

数据表	用途
user	保存用户基本信息，为账户户名和登录用户上下文提供支持。
account	保存银行账号、账户类型、余额和状态，是查余额和转账的基础表。
transaction_record	保存收入、支出、金额、对方账号、备注和交易时间，是流水查询和统计分析的数据来源。
payee_contact	保存用户收款人、银行卡号和昵称，用于 AI 按昵称转账。

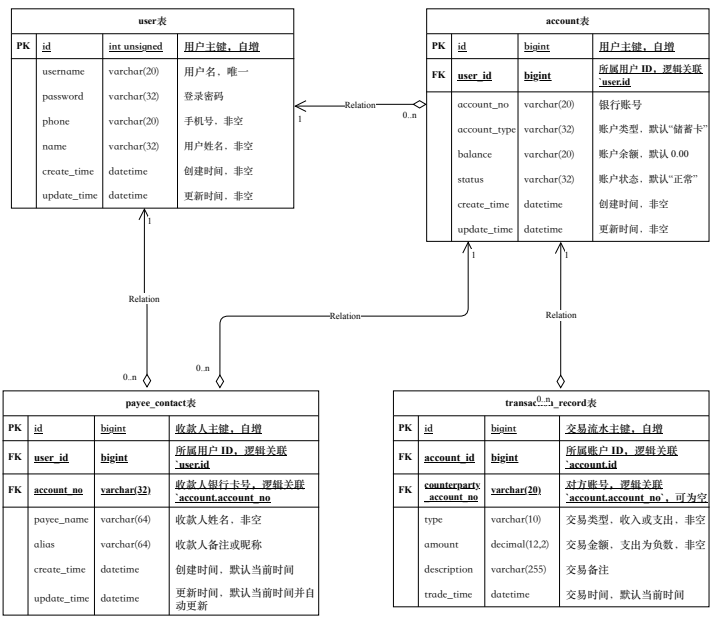


图 4.3 数据库 ER 图

4.4 接口与交互设计

AI 相关接口主要包括普通聊天接口、流式聊天接口和简报接口。普通聊天接口用于非流式请求或流式失败后的降级处理；流式聊天接口用于提升用户等待体验；简报接口用于获取周报或月报数据。

表 4.3 AI 相关接口设计表

接口	方法	功能
/assistant/chat	POST	普通 AI 聊天请求, 返回完整 AssistantChatResponse。
/assistant/chat/stream	POST	SSE 流式 AI 聊天请求, 返回 trace、message、done 等事件。
/assistant/brief	GET	获取 AI 周报或月报简报。
/transactions/export	GET	根据统计口径导出交易流水 Excel。

前端交互设计上, AI 面板既可作为首页主体展示, 也可通过右上角智能客服抽屉打开。面板顶部展示周报/月报卡片, 中间展示聊天记录和图表, 底部提供文本输入框、演示脚本按钮和发送按钮。当 AI 回复等待转账确认时, 前端显示确认和取消快捷按钮。

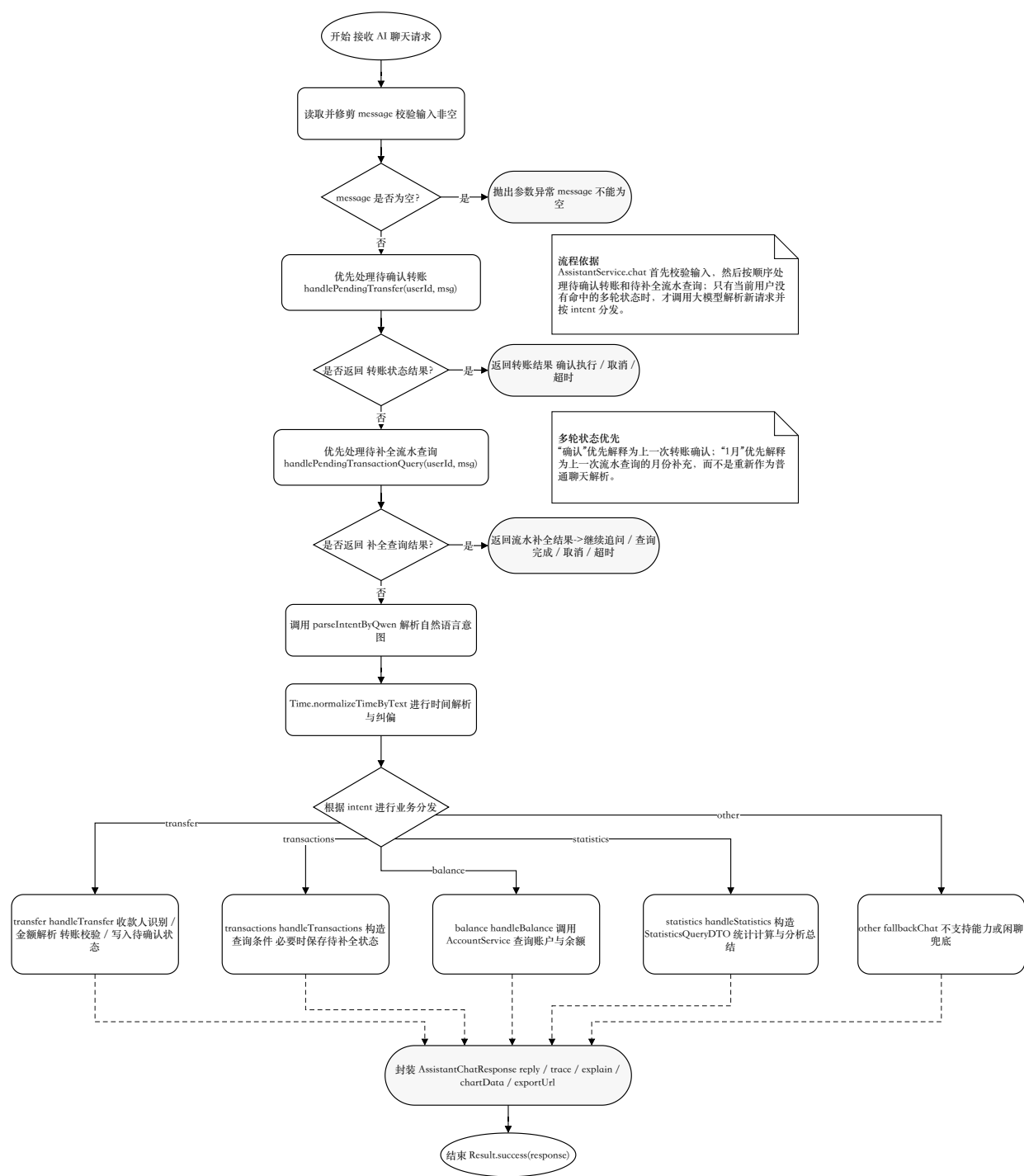
接口设计遵循前后端分离项目的基本原则。普通聊天接口适合一次性返回结果, 便于调试和兜底; 流式聊天接口适合实际交互, 可以让前端先显示处理步骤,

再显示最终回复；简报接口面向页面初始化场景，用户打开 AI 面板即可看到近期账户概览。不同接口服务于不同交互时机，避免把所有逻辑都压在单一接口中，也便于后续测试和维护。

第五章 AI 智能客服核心功能设计与实现

5.1 AI 请求处理总体流程

AssistantService 的 chat 方法是 AI 请求处理的核心入口。该方法首先校验用户输入是否为空，然后优先处理待确认转账和待补全流水查询。如果当前用户存在未完成的多轮状态，则根据用户输入继续处理该状态；若不存在，则调用 parseIntentByQwen 解析新请求的意图。解析完成后，系统根据 intent 分发到 handleTransfer、handleTransactions、handleBalance、handleStatistics 或 fallbackChat。



该流程体现了“多轮状态优先，新请求解析其次”的设计思路。例如用户发起转账后，系统等待“确认”或“取消”，此时下一条用户输入应优先作为转账确认处理，而不是重新解析为普通聊天请求。类似地，当用户查询“17 号收入记录”但缺少月份时，系统会等待用户补充月份，下一条“1 月”应被视为补全条件。

这种处理顺序符合聊天式业务系统的实现规律。单轮问答只需要根据当前输入返回结果，而银行客服机器人需要处理用户前后两轮输入之间的关系。如果不维护状态，系统就无法判断“确认”到底确认什么，也无法判断“1月”是在补充哪个查询。

在响应结构上，系统不仅返回 `reply`，还返回 `trace`、`chartData`、`exportUrl` 和 `explain` 等字段。`reply` 面向普通用户，强调可读性；`trace` 面向可解释性和调试，说明识别意图、调用工具和处理步骤；`chartData` 面向前端图表展示；`exportUrl` 面向交易流水导出；`explain` 面向统计口径说明。通过这些结构化字段，前端可以把一次 AI 回复拆分为文本、图表、过程和操作入口，使系统更像一个完整的智能业务助手，而不是简单聊天窗口。

5.2 AI 查余额功能实现

AI 查余额对应 `balance` 意图。当用户输入“查一下我的余额”“我的账户信息”等表达时，`parseIntentByQwen` 会将 `intent` 解析为 `balance`。`AssistantService` 随后调用 `handleBalance` 方法，通过 `AccountServiceImpl.getAccountInfo` 查询当前登录用户的账户信息。

`handleBalance` 返回的内容包括户名、账号、账户类型、账户状态和余额。该功能实现较为直接，但它体现了系统设计中的关键原则：AI 只识别用户想查余额，余额数据本身必须来自数据库账户表，而不是来自模型生成。

余额查询功能虽然业务逻辑相对简单，但在本论文中具有基础性意义。该功能证明 AI 助手已经能够从自然语言入口进入真实后端服务，而不是只返回固定话术。同时还为后续流水查询、统计分析和转账功能提供了账户上下文基础。若余额查询无法保证数据准确，后续涉及金额分析和资金操作的功能就缺少可信前提。因此本文将其作为核心功能之一进行说明。

表 5.1 AI 查余额功能实现说明

输入示例	解析意图	调用方法	返回内容
查一下我的余额	balance	handleBalance	户名、账号、账户类型、状态、余额。
我的账户信息	balance	handleBalance	账户基本信息和余额。

5.3 AI 查流水功能实现

AI 查流水对应 `transactions` 意图。用户可以使用自然语言指定交易类型、时间范围和条数限制。例如“查最近 3 条支出记录”会被解析为 `txnType=支出`、`limit=3`、

timeRange=latest; “查今天的流水” 会被解析为 timeRange=day; “查前几天的支出记录” 会经过 Time.normalizeTimeByText 纠偏为 recent_days。

handleTransactions 方法会根据解析结果构造 TransactionQueryDTO, 并调用 Time.fillTimeRange 填充 startTime 和 endTime。随后系统调用 TransactionServiceImpl.getTransactions 进行分页查询, 底层 SQL 由 TransactionMapper.xml 中的 selectTransactions 完成。查询结果由 convertTransactionResultToChat 格式化为自然语言列表, 并对对方账号进行脱敏处理。

对于缺少月份的日期查询, 系统设计了多轮补全机制。当 needMonth 为 true 且存在 day 但 month 为空时, 系统将当前查询保存在 pendingTxnQueryMap 中, 并提示用户补充月份。用户回复 “1 月” 后, 下一轮请求会优先进入 handlePendingTransactionQuery, 而不是被当作新的普通问题重新解析。该方法会读取待补全状态、解析月份、构造 TransactionQueryDTO, 并继续调用流水查询服务完成查询。

流水查询比余额查询更能体现自然语言到结构化条件的转换过程。用户表达中可能包含交易方向、相对时间、绝对日期、条数限制等多种信息, 后端需要把它们映射为 TransactionQueryDTO 中的字段。由于交易流水是账单统计和历史金额复用的基础数据, 查询条件的准确性直接影响后续功能结果。本文在实现中采用模型解析和规则纠偏结合的方式: 模型负责理解用户表达, Time 工具负责修正时间范围, Mapper 层负责执行确定 SQL 查询。

图5.2 以时序图形式展示了缺少月份时的两轮交互过程。第一轮中, 用户提出 “查 17 号收入记录”, 系统识别为 transactions 意图并发现 month 为空, 于是保存 PendingTransactionQuery 并追问月份; 第二轮中, 用户回复 “1 月”, AssistantService 优先命中 pendingTxnQueryMap, 补齐日期范围后调用 TransactionService 和 TransactionMapper 完成流水查询。

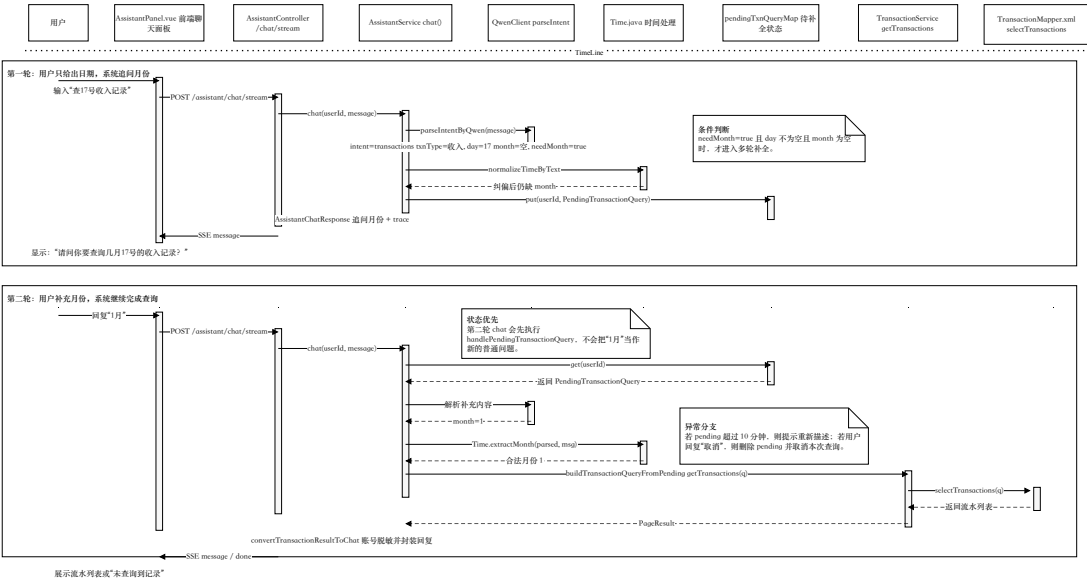


表 5.2 AI 查流水测试输入与处理方式

用户输入	关键解析字段	系统行为
查最近 3 条支出记录	txnType= 支 出, limit=3, timeRange=latest	查询最近 3 条支出流水。
查今天的流水	timeRange=day	查询当天全部流水。
查前几天的支出记录	txnType= 支 出, timeRange=recent_days, rangeValue=3	查询近 3 天支出流水。
查 17 号收入记录	txnType= 收 入, day=17, need-Month=true	追问月份并进入待补全状态。

5.4 AI 账单统计与分析功能实现

AI 账单统计与分析对应 statistics 意图。系统支持 sum、count、max 和 trend 四类统计指标，分别用于统计总金额、流水笔数、最大一笔交易和趋势数据。用户输入“统计这个月总支出”时，系统会构造 txnType= 支出、metric=sum、timeRange=month 的 StatisticsQueryDTO；用户输入“看最近 7 天支出趋势”时，系统会构造 txnType= 支出、metric=trend、timeRange=recent_days、groupBy=day 的查询对象。

StatisticsServiceImpl 根据 metric 调用不同 Mapper 方法。sumAmount 用于总额统计，countByStatistics 用于笔数统计，maxTransaction 用于最大一笔交易，statisticsTrend 用于趋势聚合。当用户查询整体收支概览时，系统还会通过

sumIncomeAndExpense、statisticsByCounterparty 生成收支对比和按对方账号聚合的图表数据。因此，账单统计与分析本质上是对交易流水查询能力的进一步聚合和解释：流水查询返回明细记录，统计分析则在同一数据基础上完成汇总、趋势和概览。

统计分析文本的生成采用“事实构造 + 模型总结 + 规则兜底”的设计。AssistantService 会将统计标题、统计类型、统计指标、时间范围、统计值、趋势明细、主要对方账号和异常信号等内容整理成 facts，然后请求通义千问生成自然语言分析总结。若模型输出为空、过于模板化或包含不支持的消费分类内容，系统会回退到规则化总结。与上一节多轮查流水不同，统计分析功能的重点不是多个参与对象之间的消息时序，而是自然语言条件如何被转换为统计 DTO，并经过确定性 SQL 聚合、事实整理和模型表达生成完整分析结果，因此本文使用流程图描述该处理管线。

图5.3 展示了 AI 账单统计与分析的整体处理流程。系统先解析 statistics 意图并构造 StatisticsQueryDTO，再由 StatisticsServiceImpl 和 TransactionMapper.xml 完成总额、笔数、最大值或趋势等确定性统计。统计结果被封装为 StatisticsResultVO 后，AssistantService 进一步整理 facts 并调用大模型生成分析总结；当模型输出不符合约束时，系统使用规则化总结兜底，最后将 reply、chartData、explain 和 exportUrl 等字段返回前端展示。

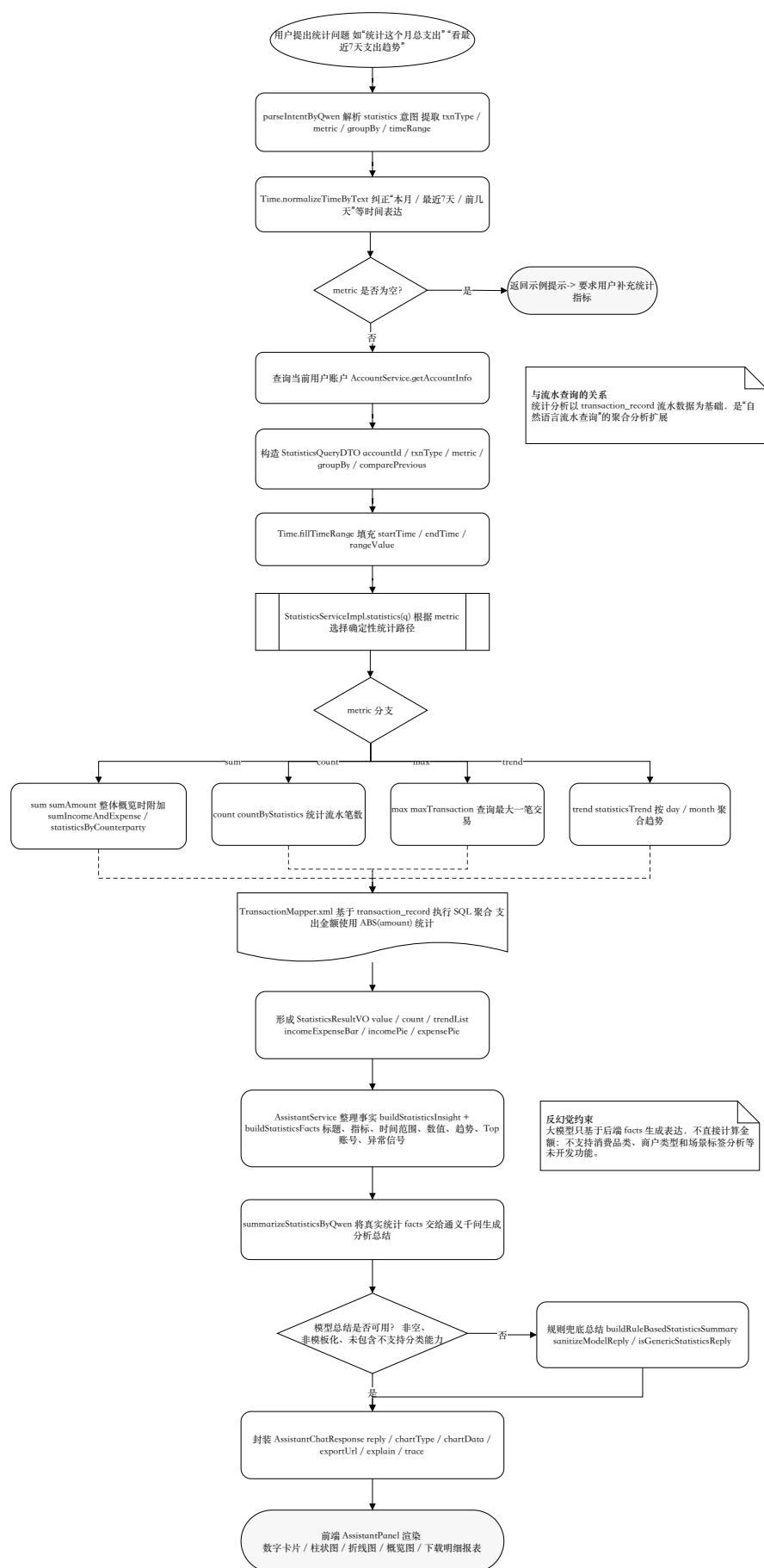


图 5.3 AI 账单统计与分析流程图

表 5.3 统计指标与实现方法对应关系

统计指标	含义	底层方法	前端展示
sum	总金额统计	sumAmount	数字结果或收支概览图。
count	笔数统计	countByStatistics	数字结果。
max	最大一笔统计	maxTransaction	数字结果。
trend	趋势统计	statisticsTrend	柱状图或折线图。

前端根据 AssistantChatResponse 中的 chartType 和 chartData 渲染图表。chartType 为 number 时显示数字卡片，为 bar 或 line 时显示趋势图，为 overview 时显示收支对比柱状图和收入、支出按对方账号聚合的饼图。系统还会生成 exportUrl，用户可以下载当前统计口径下的交易流水 Excel 文件。

5.5 AI 周报/月报功能实现

AI 周报/月报由 /assistant/brief 接口提供，主要用于用户打开 AI 面板时自动展示财务概览。该功能不依赖用户主动提问，而是根据 period 参数生成 week 或 month 简报。

getBrief 方法会先查询当前用户账户，然后构造收入总额、支出总额和支出趋势三个统计查询。系统计算总收入、总支出和净流入，并进一步生成与上周或上月的对比数据。对于异常提醒，系统使用近 4 周平均支出和近 30 天支出金额 P90 阈值作为参考，检测当前周期支出是否明显偏高或是否存在超大单笔支出。

前端 AssistantPanel 在 mounted 阶段调用 fetchAssistantBriefApi 加载简报，并在面板顶部展示 headline、highlights、suggestions、anomalies、comparison、quickActions 和 chartData。用户可以在周报和月报之间切换，也可以点击快捷追问按钮继续向 AI 发起查询。

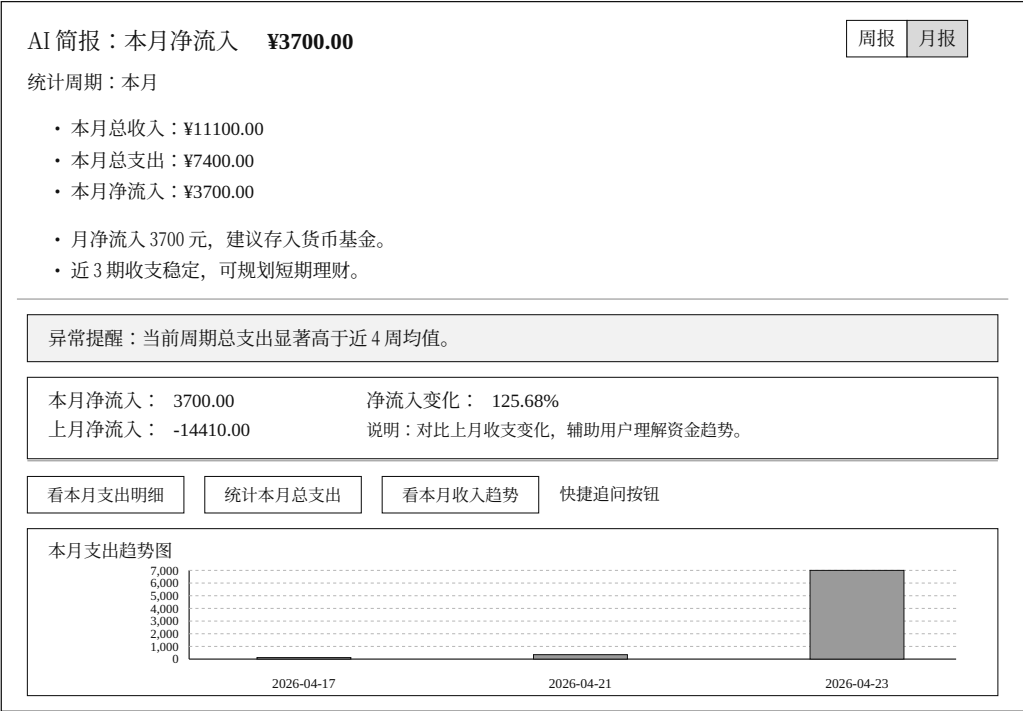


图 5.4 AI 月报卡片界面图

表 5.4 AI 周报/月报返回字段说明

简报字段	含义
headline	简报标题，例如“AI 简报：本月净流入”。
highlights	本周或本月总收入、总支出、净流入。
comparison	当前周期与上一周期的收入、支出、净流入对比。
anomalies	异常支出提醒。
suggestions	AI 生成的简短建议。
quickActions	可直接点击发送的追问指令。
chartData	支出趋势图数据。

5.6 AI 智能转账功能实现

AI 智能转账对应 transfer 意图，是系统中安全要求最高的 AI 功能。用户可以直接提供银行卡号，也可以使用收款人昵称。例如“向 6222000012345678 转 100 元”会直接提取 toAccountNo 和 amount；“给弟弟转 300 元，备注午餐费”会提取 payeeAlias= 弟弟、amount=300、description= 午餐费。

handleTransfer 方法首先解析收款账号、收款人昵称、金额、备注和 repeat-Mode。如果用户没有提供银行卡号但提供了昵称，系统会通过 PayeeContactMapper.selectByUserIdAndAlias 查询当前用户的收款人列表。若未找到昵称，系统提示用户先新增收款人或直接提供银行卡号；若匹配多个收款人，系统提示用户使

用更具体昵称。

转账金额解析完成后，系统并不会立即执行扣款，而是调用 `accountService.validateTransfer` 或 `payeeService.validateTransfer` 进行校验。校验内容包括参数完整性、金额是否大于 0、付款账户是否存在、收款账户是否存在、账户是否冻结、是否给自己转账以及余额是否充足。校验通过后，系统将转账信息写入 `PendingTransfer`，并向用户返回脱敏后的确认信息。用户回复“确认”后，`handlePendingTransfer` 才会调用真实转账服务执行扣款、收款和流水写入；用户回复“取消”则删除待确认状态。

由于 AI 智能转账涉及两轮用户交互和资金操作确认，本文使用时序图展示该过程。如图 5.5 所示，第一轮中用户通过自然语言发起转账，`AssistantService` 调用 `QwenClient` 解析 `transfer` 意图，并根据银行卡号或收款人昵称完成收款人匹配；随后系统调用转账校验服务检查账户状态、余额和自转账等风险条件。校验通过后，系统只生成确认信息并保存 `PendingTransfer`，不执行资金变动。第二轮中，用户回复“确认”时系统才调用真实转账服务更新账户余额并写入交易流水；用户回复“取消”或待确认状态超时则清理 `pendingMap`，资金操作不会发生。

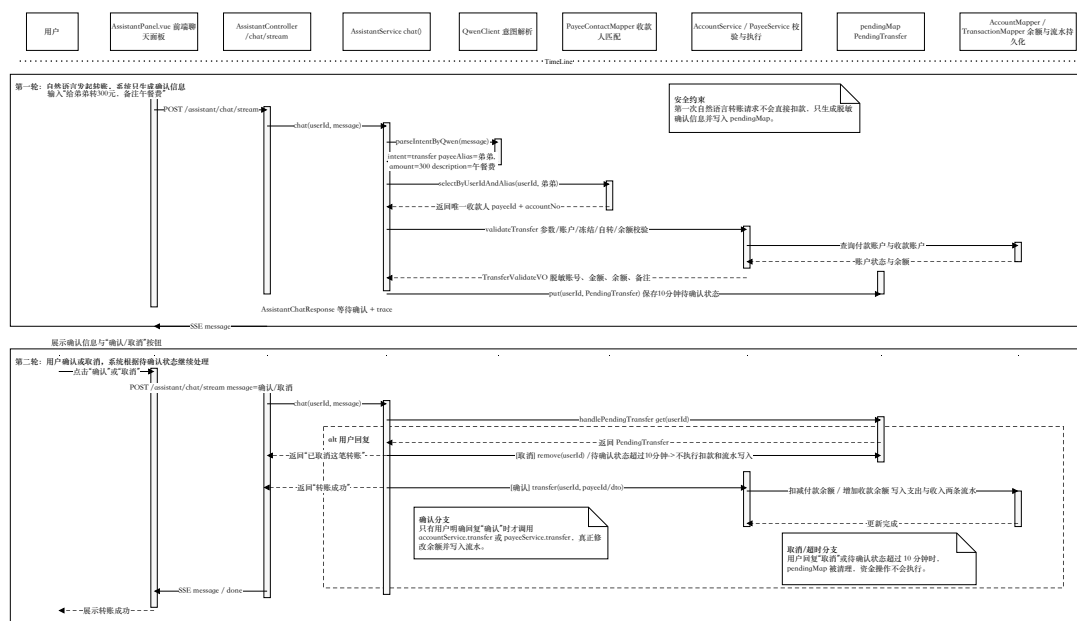


图 5.5 AI 智能转账二次确认时序图

系统还实现了历史金额复用能力。当用户说“给弟弟转晚餐费，和前天一样”时，模型会将 `repeatMode` 解析为 `time_same`，后端根据 `Time` 工具确定参考时间范围，并通过 `selectLatestExpenseByCounterpartyAndDescription` 查询该时间范围内同收款人且备注匹配的历史支出流水。若找到记录，系统复用历史金额和备注生成

确认信息；若未找到，则提示用户补充转账金额。该能力只解决用户未明确给出金额时的金额补全问题，并不会绕过转账校验和二次确认。

图5.6 展示了历史金额复用转账的处理流程。系统首先判断当前 **transfer** 请求是否属于 **amount** 为空且包含时间参考的复用场景，然后解析收款人、备注关键词和参考时间范围。查询历史流水时，系统优先查找同收款人、同备注关键词的支出记录；若未找到，再回退查询同收款人的最近支出流水。只有找到真实历史流水时，系统才复用其金额和备注，否则要求用户补充金额。

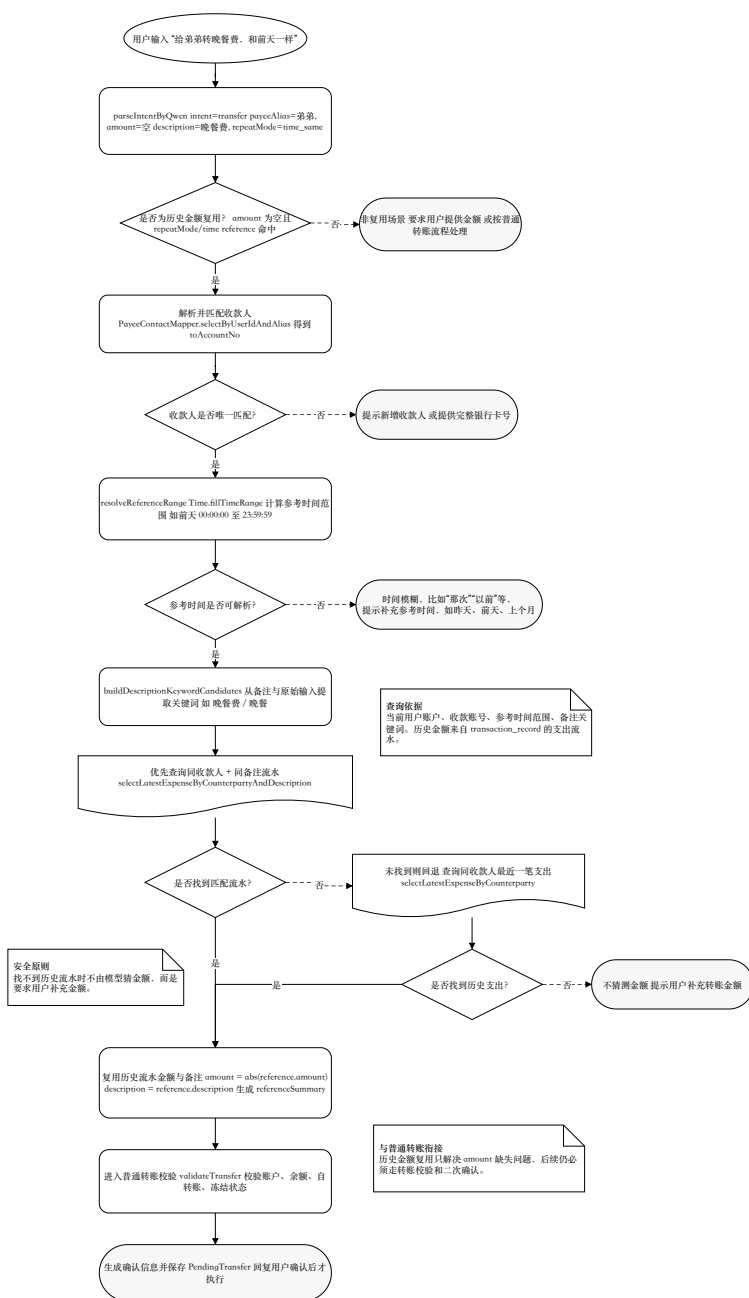


图 5.6 历史金额复用转账流程图

表 5.5 AI 智能转账安全控制设计

安全控制点	实现方式
收款人确认	按银行卡号或收款人昵称解析，昵称不存在或重复时提示用户补充。
账户校验	校验付款账户、收款账户是否存在及状态是否正常。
余额校验	转账前检查付款账户余额是否充足。
防止自转账	付款账号与收款账号相同时抛出业务异常。
二次确认	AI 首次回复只生成确认信息，用户回复确认后才执行。
账号脱敏	确认信息中付款账号和收款账号均以脱敏形式展示。

第六章 系统测试与交互优化分析

6.1 测试环境与测试方法

系统测试以本地开发环境为基础，后端服务运行在 8081 端口，前端通过 Vite 开发服务器代理 /api 请求到后端。数据库使用 MySQL，测试数据来自 sql 目录中的 user、account、payee_contact 和 transaction_record 脚本。测试方法以功能测试和交互流程测试为主，重点验证 AI 助手是否能够正确识别用户输入、调用对应业务服务、返回符合预期的文本、图表和确认流程。

由于本文是项目初稿阶段，测试结果主要以开发者功能验证为依据，尚未形成大规模真实用户调研数据。因此，交互优化分析重点从系统设计和功能可用性角度展开，不虚构用户满意度统计。

测试范围围绕本文定义的 AI 核心功能展开，不将登录、注册、普通页面跳转等基础管理能力作为主要评价对象。测试时首先保证后端服务、数据库和前端代理能够正常启动，然后在 AI 面板中输入典型自然语言指令，观察后端控制台、浏览器界面和接口返回结果是否一致。对于查询类功能，重点检查返回金额、记录条数、交易方向和时间范围是否与数据库样本相符；对于统计类功能，重点检查统计口径、聚合结果、图表数据和导出链接是否完整；对于转账类功能，重点检查系统是否先生成确认信息，是否在用户明确确认前避免直接修改账户余额。

为保证测试结论与源代码一致，本文没有设计超出项目能力范围的测试项。例如，当前系统并未实现商产品类识别和消费场景标签，因此测试只验证系统能否正确拒绝或说明该类请求，而不把这类功能写成已经完成的能力。对于大模型输出具有一定随机性的情况，测试判断以结构化字段和业务服务调用结果为主，而不是要求每次自然语言回复完全相同。这样可以避免把模型措辞差异误判为功能失败，也能更准确地反映系统的真实可用性。

6.2 功能测试

表 6.1 AI 核心功能测试用例表

编号	测试输入或操作	预期结果	结论
T1	输入“查一下我的余额”	识别为 balance ，返回账户信息和余额。	通过
T2	输入“查最近 5 条支出记录”	识别为 transactions ，返回支出流水列表。	通过
T3	输入“查 17 号收入记录”后回复“1 月”	系统先追问月份，再查询指定日期收入流水。	通过
T4	输入“统计这个月总支出”	识别为 statistics ，返回支出总额、统计口径和导出链接。	通过
T5	输入“看最近 7 天支出趋势”	返回趋势图数据，前端可渲染柱状图或折线图。	通过
T6	打开 AI 面板	自动加载周报或月报卡片。	通过
T7	输入“给弟弟转 300 元，备注午餐费”	按昵称匹配收款人并返回转账确认信息。	通过
T8	在待确认转账状态下输入“取消”	取消转账并清除待确认状态。	通过
T9	提出消费分类或商户类型分析请求	系统说明当前不支持该类能力。	通过

从测试结果看，系统能够覆盖论文设计的五项核心 AI 功能。对于常规查询类任务，系统能够通过意图解析进入对应业务流程；对于缺失信息的查询，系统能够通过多轮对话补全；对于转账类任务，系统能够在执行前给出确认信息并等待用户明确操作。

余额查询测试主要验证账户数据读取链路。用户输入余额查询请求后，后端将其解析为 **balance** 意图，并调用账户相关服务读取当前用户账户信息。测试中需关注两点：一是回复内容不能脱离数据库真实余额自行生成；二是当账户不存在或数据异常时，系统应返回明确提示，而不是给出含糊的正常结果。该测试说明余额查询功能已经具备作为智能客服基础能力的条件。

流水查询测试主要验证自然语言条件到查询参数的映射。系统需要从“最近 5 条支出记录”“17 号收入记录”等表达中识别交易方向、数量限制和日期信息。对于日期不完整的表达，系统不会直接猜测月份，而是进入多轮补全流程，要求用户进一步说明。该处理方式比直接推断更稳妥，能够降低查询范围错误导致的用户误解。测试结果表明，系统可以在常见流水查询表达下生成可执行条件，并正确返回流水列表。

账单统计与分析测试主要验证数据库聚合和图表数据生成。系统接收到“统

计这个月总支出”“最近 7 天支出趋势”等请求后，会构造 `StatisticsQueryDTO`，并由后端统计服务完成聚合计算。大模型不直接计算金额，而是在后端已经形成事实数据的基础上生成说明性文本。测试中，如果统计结果为空，系统也需要说明当前范围内没有相关记录。该设计符合银行业务对数据真实可靠的要求。

周报/月报测试主要验证系统主动概览能力。用户打开 AI 面板后，前端会请求简报数据，并将收入、支出、净流入、趋势和快捷追问组织为卡片。该能力的价值在于用户不必主动输入复杂问题，也能快速了解近期账户变化。测试时重点检查简报统计周期是否正确、卡片数据是否完整、快捷追问是否能够继续进入 AI 聊天流程。

智能转账测试是安全性要求最高的部分。系统需要识别收款人、金额和备注，并根据常用收款人信息匹配真实收款账号。首次回复只展示待确认信息，用户输入确认或点击确认按钮后才会调用转账服务。测试结果表明，系统能够处理确认和取消两类分支，并在取消后清除待确认状态。对于缺失金额的参考转账请求，系统会从历史流水中尝试查找相同收款人和备注关键词对应的金额，但最终仍需要用户确认后执行。

6.3 交互优化设计分析

第一，系统采用流式响应降低等待感。前端调用 `/assistant/chat/stream` 后，后端先返回 `trace` 事件，提示“请求已接收”和“正在识别业务意图”，最终再返回 `message` 事件。相比普通请求一次性返回结果，流式处理能让用户知道系统正在处理请求，减少等待过程中的不确定性。

第二，系统展示 AI 处理过程提升可解释性。`AssistantChatResponse` 中的 `trace` 字段包含 `intent`、`tool`、`status`、`confidence` 和 `steps`。前端将这些信息展示在聊天气泡中，使用户和答辩评阅者能够看到系统识别了什么意图、调用了什么能力以及处理状态。

第三，系统通过图表增强统计结果表达。对于趋势类统计，用户不仅能看到文字总结，还能看到柱状图或折线图。对于收支概览，系统能够展示收支对比和按对方账号聚合的饼图。图表展示降低了用户理解复杂流水数据的成本。

第四，系统提供快捷操作减少输入成本。AI 面板内置演示脚本按钮，便于展示时快速输入典型问题；简报卡片提供快捷追问按钮，用户可以一键继续查询；转账确认场景中显示“确认”和“取消”按钮，减少用户手动输入。

第五，系统对不支持能力进行边界控制。当前项目并未实现消费品类、商户类型和场景标签分析，因此后端在意图解析后会对这些需求进行拦截，避免大模

型给出超出系统能力的结论。这种设计有利于提高银行场景下的可信度。

需要展示聊天气泡中的意图、能力、状态、置信度和处理步骤。建议文件名：**fig-trace-ui.pdf**

图 6.1 AI 处理过程展示界面图

需要展示趋势图、收支对比图和报表下载按钮。建议文件名：**fig-statistics-ui.pdf**

图 6.2 AI 统计图表展示界面图

6.4 测试结论

综合功能测试和交互分析，BankCopilot 智能客服机器人能够较好地完成文本自然语言形式的银行业务交互。系统在查询类任务中能够正确调用账户和流水服务，在统计类任务中能够基于数据库聚合结果生成图表和分析，在转账类任务中能够通过二次确认保证资金操作安全。虽然系统仍处于原型阶段，但其整体设计体现了智能客服机器人在银行场景中的可行性和实用价值。

从测试结论看，本文系统已经完成了从自然语言输入到业务服务调用、再到结构化结果展示的闭环。其主要优势体现在三个方面：第一，系统没有把银行业

务结果完全交给大模型生成，而是以真实数据库和后端服务为依据；第二，系统对查询、统计和转账采用不同交互策略，能够根据业务风险调整流程复杂度；第三，系统在展示层面保留 AI 处理过程和统计口径说明，便于用户理解系统为何给出某一结果。

同时，测试也反映出当前系统仍属于教学展示和原型验证阶段。测试数据规模较小，尚不能覆盖真实银行系统中复杂的账户状态、异常交易、并发转账和风控规则。后续如果将系统扩展到更真实的业务环境，还需要补充自动化测试、压力测试、安全测试和异常恢复测试。本文在结论中将这些问题作为不足进行说明，避免夸大系统当前能力。

第七章 结论与展望

7.1 主要结论

本文围绕“智能客服机器人的设计与交互优化”这一课题，设计并实现了 BankCopilot 银行智能客服机器人原型。系统以文本自然语言交互为入口，通过通义千问大模型完成意图解析和分析性回复生成，并通过 Spring Boot 后端服务完成真实账户查询、流水查询、统计计算和转账执行。

在功能实现方面，系统完成了 AI 查余额、AI 查流水、AI 账单统计与分析、AI 周报/月报和 AI 智能转账五项核心能力。AI 查余额和查流水满足用户高频查询需求；账单统计与分析将数据库聚合结果转化为文本总结和图表；周报/月报实现主动式财务概览；智能转账实现自然语言转账、昵称匹配、历史金额复用和二次确认。

在交互优化方面，系统实现了 SSE 流式响应、AI 处理过程展示、统计口径解释、图表渲染、报表导出、快捷追问和确认按钮等设计。这些优化使智能客服不仅能够回答问题，还能够以更清晰、更可控的方式帮助用户完成银行业务。

综合需求分析、系统设计、功能实现和测试结果，可以得到以下结论。首先，将大语言模型定位为自然语言理解与表达增强组件，而不是直接业务决策组件，是银行智能客服系统中较为稳妥的实现方式。其次，余额、流水、统计和转账等业务必须通过后端确定性服务完成，才能保证数据来源清楚、结果可追溯。再次，智能客服的交互优化不仅体现为回复更自然，还体现为过程可见、结果可解释、风险操作可确认。最后，对于本科毕业设计项目而言，BankCopilot 已经形成较完整的 AI 功能闭环，能够支撑课堂展示和论文论证。

7.2 实践价值

BankCopilot 的实践价值主要体现在智能客服能力与银行业务流程的结合。传统页面更适合结构化操作，但当用户目标明确而路径复杂时，自然语言入口可以减少页面查找和条件设置成本。例如，用户只需要输入“查最近 5 条支出记录”，系统即可完成意图识别、条件转换、流水查询和结果展示。对于账单统计，系统能够把分散流水转化为总额、趋势和概览，使用户更容易理解账户变化。

在安全性方面，本文系统没有因为引入 AI 而弱化业务控制。尤其在转账功能中，系统把自然语言解析结果转化为待确认信息，并在确认前保持状态而不执行

资金变动。这种方式既保留了智能交互的便利性，又符合资金类业务需要用户明确授权的原则。对于毕业设计展示而言，该功能能够较直观地体现“智能”并不等同于自动越权执行，而是要在用户授权、业务规则和交互体验之间取得平衡。

在可解释性方面，系统通过 **trace** 信息展示意图、工具、状态、置信度和处理步骤。该设计使 AI 处理过程从黑盒回复变为可观察链路，便于用户理解系统处理逻辑，也便于教师在评阅时确认系统不是简单静态问答。对于账单分析和统计图表，系统同时给出统计口径说明，避免用户把不同时间范围、不同交易方向或不同聚合方式下的结果混淆。

7.3 不足与展望

当前系统仍存在一些不足。首先，多轮对话状态保存在后端内存中，服务重启后会丢失，后续可引入 **Redis** 实现会话状态持久化。其次，系统依赖外部大模型服务，当网络异常或模型服务不可用时，意图解析和分析总结能力会受到影响，后续可增加本地规则兜底或缓存机制。再次，当前系统不支持消费品类、商户类型和场景标签等更细粒度分析，后续可通过扩展流水数据字段和分类模型实现更深入的财务分析。

后续研究还可以从操作审计、风险控制和个性化服务三个方向展开。对于涉及资金的 AI 操作，可以记录更完整的操作链路和审计日志；对于异常支出，可以结合用户历史行为建立风险提醒模型；对于账单分析，可以结合预算阈值和用户偏好给出更个性化的建议。通过这些优化，智能客服机器人可以进一步从“可用的业务助手”发展为“可信的个人金融服务助手”。

结束语

本文围绕个人银行业务中的智能客服交互问题，完成了从需求分析、总体设计、核心功能实现到测试分析的完整论述。论文写作过程中始终以现有源代码为依据，没有将项目未完成的功能写入系统成果。系统的重点不在于构建一个泛化聊天机器人，而在于验证大语言模型能否与确定性银行业务服务结合，形成可解释、可控制、可展示的智能客服原型。

从实现结果看，BankCopilot 已经能够处理余额查询、流水查询、账单统计、周报/月报和智能转账等典型场景。其设计经验说明，在银行这类高准确性要求场景中，AI 模块应当服务于意图理解、信息组织和交互优化，而关键业务数据仍应由数据库和后端服务提供。本文仍存在测试规模有限、风控规则较简单、会话状态未持久化等不足，但这些问题也为后续优化提供了明确方向。

通过本课题的完成，本人对前后端分离开发、数据库建模、自然语言意图识别、流式交互和论文规范写作有了更系统的认识。后续若继续完善该系统，可以围绕真实业务数据规模、自动化测试、安全审计和个性化分析继续展开，使其从毕业设计原型进一步向更完整的智能金融服务系统演进。

致谢

本论文和系统原型的完成离不开指导教师在选择方向、系统设计和论文写作方面的指导。在项目实现过程中，相关课程学习和开源技术文档为系统架构设计、前后端开发和数据库实现提供了重要帮助。同时，感谢在毕业设计过程中给予建议和支持的老师、同学和企业导师。由于本人能力和时间有限，论文和系统仍存在不足之处，恳请各位老师批评指正。

参考文献

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [3] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie, and J.-R. Wen, “A survey of large language models,” *arXiv preprint arXiv:2303.18223*, 2023.
- [4] Spring Team, *Spring Boot Reference Documentation*, VMware, 2025.
- [5] Vue.js Team, *Vue.js Guide*, Vuejs.org, 2025.
- [6] Apache ECharts Team, *Apache ECharts Documentation*, Apache Software Foundation, 2025.
- [7] MyBatis Team, *MyBatis 3 User Guide*, MyBatis.org, 2025.

附录 A 主要接口与数据结构

A.1 AI 聊天请求结构

AI 聊天请求由 AssistantChatRequest 承载，核心字段为 message。用户自然语言输入会以 JSON 请求体形式发送至 /assistant/chat 或 /assistant/chat/stream 接口。

表 A.1 AssistantChatRequest 请求字段

字段	类型	说明
message	String	用户自然语言输入，例如“统计这个月总支出”。

A.2 AI 聊天响应结构

表 A.2 AssistantChatResponse 响应字段

字段	类型	说明
reply	String	AI 回复文本。
chartType	String	图表类型，如 number、bar、line、overview。
chartData	Object	图表数据或数字统计结果。
exportUrl	String	导出交易流水的链接。
trace	TraceInfo	AI 处理过程信息。
explain	Map	查询或统计口径说明。

A.3 统计查询结构

表 A.3 StatisticsQueryDTO 字段说明

字段	说明
accountId	账户 ID。
txnType	收入或支出。
metric	统计指标，支持 sum、count、max、trend。
groupBy	分组方式，支持 none、day、month。
startTime	统计开始时间。
endTime	统计结束时间。
rangeValue	最近 N 天或 N 年的数值。
comparePrevious	是否对比上一周期。